



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



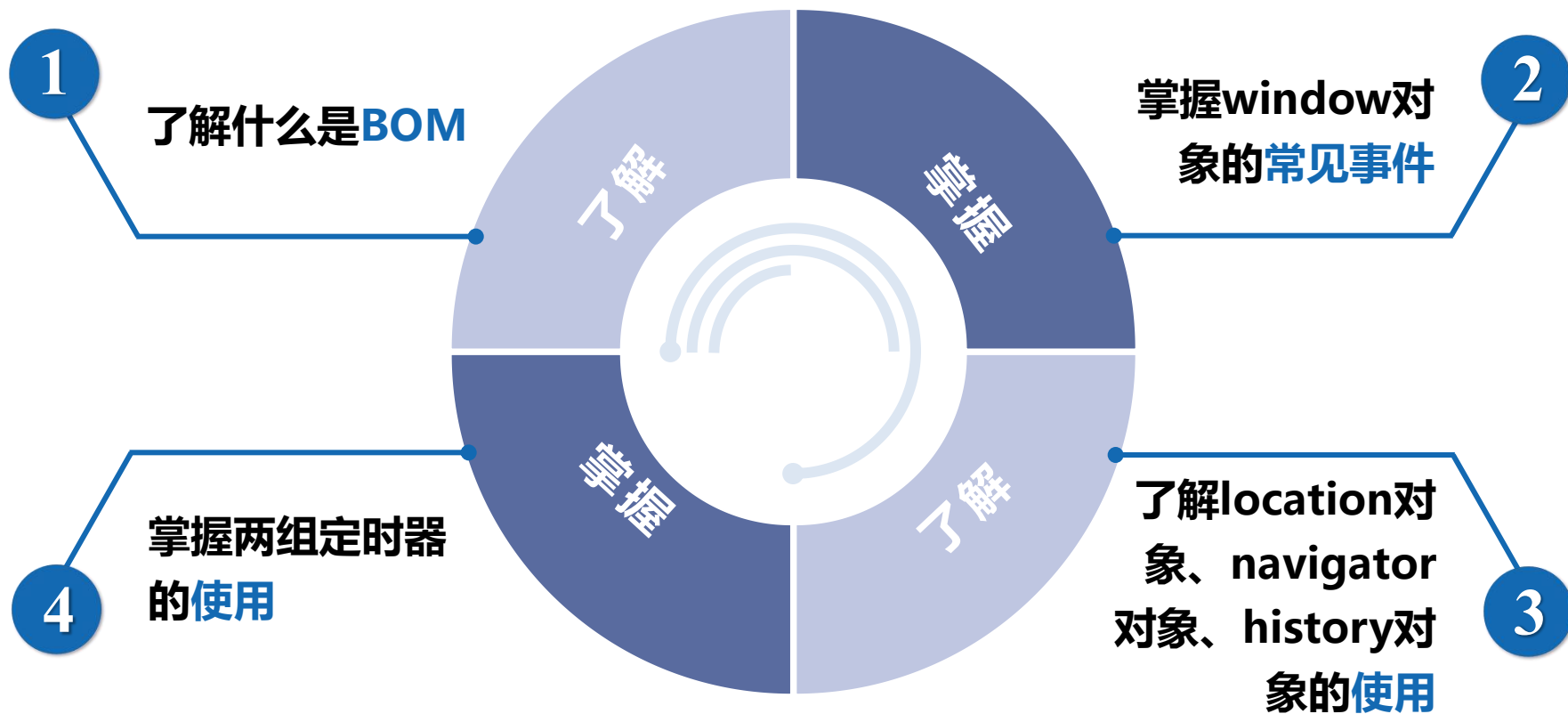
内容回顾1

- 节点：元素、属性、文本
- 节点的层级：根、父子、兄弟
 - 获取父级节点：obj.parentNode
 - 获取子级节点：childNodes, children, firstChild和lastChild, firstElementChild和lastElementChild
 - 获取兄弟节点：nextSibling和previousSibling, nextElementSibling和previousElementSibling
- 节点操作：
 - 创建节点：document.write(), element.innerHTML, document.createElement()
 - 添加节点：node.appendChild()和node.insertBefore()
 - 删除节点：node.removeChild(child)
 - 复制节点：需要被复制的节点.cloneChild(true/false), 浅拷贝和深拷贝



第8章 BOM

学习目标



目录



8.1

BOM简介

👉 [点击查看本节相关知识点](#)

8.2

window对象的常见事件

👉 [点击查看本节相关知识点](#)

8.3

定时器

👉 [点击查看本节相关知识点](#)

8.4

JavaScript执行机制

👉 [点击查看本节相关知识点](#)

目录



location对象

👉 [点击查看本节相关知识点](#)



navigator对象

👉 [点击查看本节相关知识点](#)



history对象

👉 [点击查看本节相关知识点](#)



8.1 BOM简介

1

什么是BOM

BOM: 浏览器对象模型（**Brower Object Model, BOM**）提供了独立于内容而与浏览器窗口进行交互的对象，其核心对象是window。



8.1 BOM简介

2

BOM与DOM的区别

区别:

- ❑ DOM是文档对象模型，把文档当作一个对象来看待，它的顶级对象是 **document**。
- ❑ BOM是浏览器对象模型，是把浏览器当做一个对象来看待，它的顶级对象是 **window**。

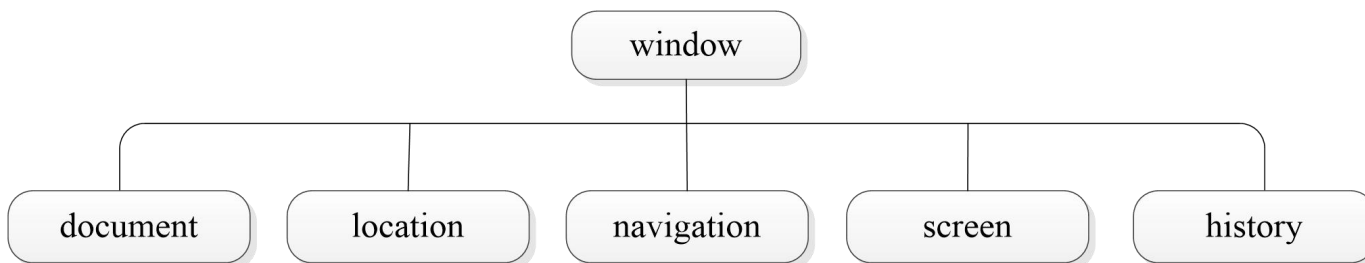


8.1 BOM简介

3

BOM的构成

BOM构成:





8.2 window对象的常见事件

1

窗口加载事件

window.onload: 是窗口（页面）加载事件，当文档内容（包括图像、脚本文件、CSS文件等）完全加载完成会触发该事件，调用该事件对应的事件处理函数。

window.onload

// 方式1

```
window.onload = function () {};
```

// 方式2

```
window.addEventListener('load',  
function () {});
```



8.2 window对象的常见事件

1

窗口加载事件

document.DOMContentLoaded : document.DOMContentLoaded加载事件，会在DOM加载完成时触发，这里所说的加载不包括CSS样式表、图片和flash动画等额外内容的加载。



8.2 window对象的常见事件

2

调整窗口大小事件

例8.2.2-1

例8.2.2-2

window.onresize: 当调整window窗口大小的时候，就触发window.onresize事件，调用事件处理函数。

window.onresize

// 方式1

```
window.onresize = function () {};
```

// 方式2

```
window.addEventListener('resize',  
function () {});
```



8.3 定时器

1

定时器方法

定时器方法如下：

方法	说明
setTimeout()	在指定的毫秒数后调用函数或执行一段代码
clearTimeout()	取消由setTimeout()方法设置的定时器
setInterval()	按照指定的周期（以毫秒计）来调用函数或执行一段代码
clearInterval()	取消由setInterval()设置的定时器



8.3 定时器

2

【案例】 setTimeout

例8.3.1-1

```
<script>
// 参数形式1：用字符串表示一段代码
setTimeout('alert("JavaScript");', 3000);

// 参数形式2：传入一个匿名函数
setTimeout(function () {
    alert('JavaScript');
}, 3000);

// 参数形式3：传入函数名
setTimeout(fn, 3000);
function fn() {
    console.log('JavaScript');
}
</script>
```



8.3 定时器

2

【案例】 setTimeout

例8.3.1-2

```
<script>
  function fn() {
    console.log('JavaScript');
  }
  // 在设置定时器时，保存定时器的唯一标识
  var timer = setTimeout(fn, 3000);
  // 如果要取消定时器，将唯一标识传递给
clearTimeout()方法
  clearTimeout(timer);
</script>
```



8.3 定时器

2

【案例】 3s后自动关闭广告

例8.3.2

使用**setTimeout()**实现3秒后自动关闭广告的效果。案例代码如下：

```
<body>
  <script>
    var timer = setTimeout(fn, 3000);
    function fn() {console.log('广告关闭了');}
  </script>
</body>
```




8.3 定时器

3

【案例】60s内只能发送一次短信

例8.3.3

实现思路:

- ① 在页面中放一个文本框和一个“发送”按钮。
- ② 在文本框中输入手机号码，然后单击“发送”按钮，就可以发送短信。
- ③ 在按钮单击之后，按钮上的文字会变为“还剩x秒再次单击”。

手机号码：



8.3 定时器

3

【案例】60s内只能发送一次短信

实现代码如下：

```
var btn = document.querySelector('button');  
var time = 60; // 定义剩下的秒数  
btn.addEventListener('click', function () {  
    btn.disabled = true;  
    //此处添加timer定时器;  
})
```



8.3 定时器

3

【案例】60s内只能发送一次短信

timer定时器:

```
var timer = setInterval(function () {  
    if (time == 0) {  
        clearInterval(timer);  
        btn.disabled = false;  
        btn.innerHTML = '发送';  
    } else {  
        btn.innerHTML = '还剩下' + time + '秒';  
        time--;  
    }  
}, 1000);
```



8.4 JavaScript执行机制

1

单线程

单线程：JavaScript语言的一大特点就是单线程，也就是说，同一个时间只能做一件事。

```
console.log(1); //先输出了1、2
setTimeout(function () {
  console.log(3); //，等待5秒后输出3
}, 5000);
console.log(2);
```



8.4 JavaScript执行机制(自学)

2

同步和异步

同步：所谓同步，就是前一个任务结束后再执行后一个任务，程序的执行顺序与任务的排列顺序是一致的、同步的。

异步：所谓异步，就是在做一件事的同时，可以去处理其他的事情。

例8.4.1



8.4 JavaScript执行机制

3

执行机制

执行栈和任务队列如下图：

执行栈

```
console.log(1)
setTimeout(fn1, 0)
console.log(2)
setTimeout(fn2, 0)
```

任务队列

fn1

fn2



8.4 JavaScript执行机制

3

执行机制

例8.4.3

通过代码演示：

```
console.log(1);
setTimeout(function () {
  console.log(3);
}, 0);
for (var i = 0, str = ""; i < 900000; i++) {
  str += i; // 利用字符串拼接运算拖慢执行时间
}
console.log(2);
```



8.5 location对象(自学)

1

URL的组成

URL组成:

各部分	说明
protocol	网络协议, 常用的如http, ftp, mailto等
host	服务器的主机名, 如www.example.com
port	端口号, 可选, 省略时使用协议的默认端口, 如http默认端口为80
path	路径, 如 "/web/index.html"
query	参数, 键值对的形式, 通过 "&" 符号分隔, 如 "a=3&b=4"



8.5 location对象

2

location常用属性

location常用属性:

属性	说明
<code>location.search</code>	返回（或设置）当前URL的查询部分（“?” 之后的部分）
<code>location.hash</code>	返回一个URL的锚部分（从“#” 开始的部分）
<code>location.host</code>	返回一个URL的主机名和端口
<code>location.hostname</code>	返回URL的主机名
<code>location.href</code>	返回完整的URL
<code>location.pathname</code>	返回URL的路径名
<code>location.port</code>	返回一个URL服务器使用的端口号
<code>location.protocol</code>	返回一个URL协议



8.5 location对象

3

【案例】获取URL参数

login.html
index.html

创建login.html登录页面:

```
<form action="index.html">  
  用户名: <input type="text" name="uname">  
  <input type="submit" value="登录">  
</form>
```



8.5 location对象

3

【案例】获取URL参数

创建index.html首页:

```
console.log(location.search); // 结果为: ?uname=andy
// 1. 去掉search中的问号 “?”
var params = location.search.substr(1);
console.log(params); // 结果为: uname=andy
// 2. 把字符串分割为数组
var arr = params.split('=');
console.log(arr); // 结果为: ["uname", "andy"]
var div = document.querySelector('div');
// 3. 把数据写入div中
div.innerHTML = arr[1] + '欢迎您';
```



8.5 location对象

4 location的常用方法

常用方法：

方法	说明
assign()	载入一个新的文档
reload()	重新加载当前文档
replace()	用新的文档替换当前文档，覆盖浏览器当前记录



8.6 navigator对象(自学)

User Agent中文名为用户代理，简称 **UA**，它是一个特殊字符串头，使得服务器能够识别客户使用的操作系统及版本、**CPU** 类型、浏览器及版本、浏览器渲染引擎、浏览器语言、浏览器插件等。

1

常用属性

属性	说明
appName	返回浏览器的内部名称
appName	返回浏览器的完整名称
appVersion	返回浏览器的平台和版本信息
cookieEnabled	返回指明浏览器中是否启用Cookie的布尔值
platform	返回运行浏览器的操作系统平台
例8.6 userAgent	返回由客户端发送到服务器的User-Agent头部的值

常用方法**javaEnabled()**: 指定是否在浏览器中启用Java。



8.7 history对象(自学)

1

常用属性

history.length属性：返回历史列表中的网址数。

2

常用方法

例8.7

方法	说明
back()	加载history列表中的前一个URL
forward()	加载history列表中的下一个URL
go()	加载history列表中的某个具体页面

本章总结



本章首先介绍了**BOM**是JavaScript组成的一部分，讲解了**BOM**的构成，以及其各属性的作用。然后通过案例的形式讲解了定时器的应用，重点讲解了**window**对象、**location**对象、**history**对象的定义、常用的属性和方法。通过本章的学习，读者可以使用**BOM**对象中的属性和方法实现窗口和**URL**导航及定时器的相关操作。